Software-Qualitätskonzept

Funktionalität

- Methoden sollen die richtigen Ausgaben und Operationen durchführen. Klassen sollen die nötige Funktionalität bereitstellen. Um dies zu überprüfen, benutzen wir White-Box- und Unit-Tests.
- Methoden und Algorithmen werden so gewählt, dass sie die Logik des Programms so einfach wie möglich halten und möglichst effizient sind. Z.b. Benutzung von Whitespaces als Trennzeichen.
- Unsere Software kann auf jedem Betriebssystem ausgeführt werden, wobei systemabhängige Funktionen auf Windows, MacOS und Linux funktionieren.
- Zugriff auf kritische Variablen ist nur durch Klassenaufrufe möglich und nur die im Netzwerkprotokoll deklarierten Befehle können vom Client und Server erstellt, gesendet und empfangen werden.

Effizienz

- Client und Server antworten innert wenigen Millisekunden. GUI reagiert ohne erkennbare Verzögerung auf Benutzereingaben. Dies wird mit Unit-Tests getestet.
- Der Client beansprucht nur so lange Ressourcen wie er mit dem Server verbunden ist.

Zuverlässigkeit

- Alle Exceptions werden mit try-catch blöcken abgefangen. Unvorhergesehene Ereignisse wie ein Verbindungsverlust werden vom Programm antizipiert und richtig verarbeitet.
- Fehlbenutzung der Software oder Auftreten von Fehlern in Hard- und Software soll möglichst verhindert werden. Dies wird schlussendlich durch Peer Reviews und Black-Box Testing geprüft.
- Der Spielstand wird auf dem Server wie auf dem Client gespeichert, um eine Wiederverbindung und Fortsetzung des Spiels nach einem Verbindungsverlust zu garantieren, sofern der Client sich wieder einloggt.

Benutzbarkeit

- Das GUI soll dem Benutzer erklären, wie das Spiel funktioniert. Die Spielregeln sollen für den Benutzer verständlich und in einem sorgfältig ausgearbeiteten Benutzerhandbuch nachlesbar sein. Die Intuitivität des Spiels wird durch Peer Reviews festgestellt.

- Der Benutzer soll das GUI intuitiv verstehen und benutzen können. Das Spiel soll mit der Dokumentation und einigen Spielrunden direkt klar sein. Ähnlich soll auch die Intuitivität des GUIs durch Peer Reviews festgestellt werden.
- Der Aufwand zur Bedienung wird durch das GUI als Schnittstelle sehr gering gehalten. Auch dies soll über Peer Reviews festgestellt werden.

Änderbarkeit

- Der Code ist mit Javadoc dokumentiert und jede Methode ist beschrieben. Fehler in den verschiedenen Subkomponenten der Software werden durch den Einsatz von Log4j getrackt. Das Programm soll so ausgiebig wie möglich mit Unit-Tests abgedeckt sein, wobei möglichst jede Methode und jede Interaktion zwischen den Klassen getestet werden soll.
- Methoden können jederzeit geändert werden. Die Weiterverwendung sollte ohne grosse Änderungen anderer Klassen möglich sein. Dafür ist es nötig, Client, Server und Spiellogik sauber voneinander zu trennen. Die Spiellogik wird in verschiedene Klassen implementiert, die die spezifische Funktionalität sinnvoll abbilden.
- Durch rigoroses Testen soll die Wahrscheinlichkeit für unerwartete Ereignisse minimiert werden. Jede Änderung sollte einer Fehleranalyse unterzogen werden.

Übertragbarkeit

- Da die JVM auf jedem Betriebssystem funktioniert, ist das Spiel an jedes Betriebssystem angepasst.
- Aufwand betrifft das Installieren der JVM sowie der Jar-Datei. Mit Hilfe letzterem kann das Spiel auf jedem Betriebssystem ausgeführt werden.

Programme

- Javadoc soll verwendet werden, um den Code zu dokumentieren und für Teammitglieder verständlich zu machen.
- Log4J soll Debugging, Fehlersuche und die Fehleranalyse erleichtern.
- JUnit soll für alle Unit-Tests benutzt werden. Dabei soll die Funktionalität der Klassen sowie die Interaktion getestet werden.
- Jacoco soll benutzt werden, um zu testen, wie viel Quellcode von UnitTests abgedeckt wird.
- Der IntelliJ Profiler soll Bottlenecks ausfindig machen und die Effizienz von Methoden und die Speicherverteilung ersichtlich machen.
- MetricsReloaded soll Metriken für die Komplexitätsanalyse des Spiels bereitstellen.

Methoden

- Unit-Tests und White-Box-Testing, um einzelne Methoden zu testen und zu debuggen.
- Gegenseitige Code Review, um den Code auf Fehler zu untersuchen.
- Peer Review, um das Spiel von aussen zu testen.